

Live Coding:

- Ergänze die Klasse **FileSystemElement** mit folgender Methode:
- `abstract Collection<FileSystemElement> elements();`
- Implementiere die Methode für die Klasse `File` und `Directory`.
- Die Methode `elements` gibt eine **unmodifiable Collection** zurück, die das Element selbst und alle enthaltenen Unter-Elemente enthält.

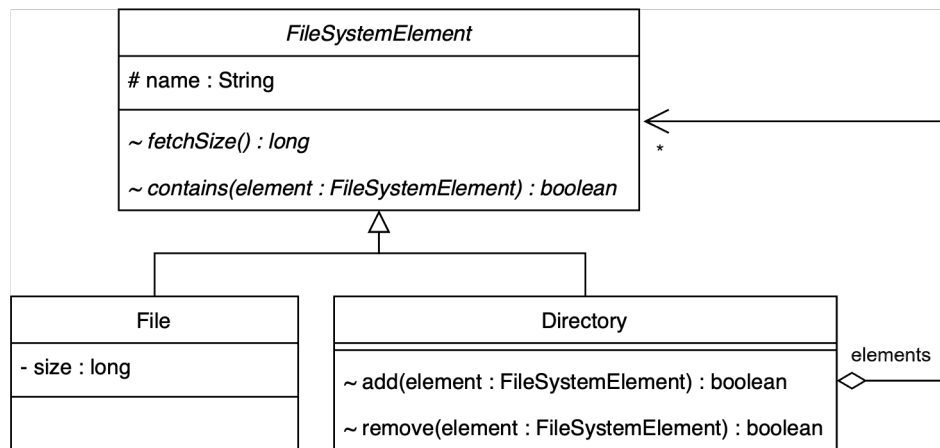
32

Kontext: Wir kennen nun das Composite Pattern.

Also zu einer Übungsaufgabe:

Die Klassen `FileSystemElement`, `File` und `Directory` existieren bereits.

Bestehenden Code erklären



33

Tests zeigen

```
@Test
void dir1Elements() {
    var actual = dir1.elements();
    var expected = List.of(dir1, file1, dir2, dir3, file2, file3, dir4);
    assertIterableEquals(expected, actual);
}

@Test
void dir3Elements() {
    var actual = dir3.elements();
    var expected = List.of(dir3, file2, file3);
    assertIterableEquals(expected, actual);
}

@Test
void file1Elements() {
    var actual = file1.elements();
    var expected = List.of(file1);
    assertIterableEquals(expected, actual);
}
```

34

Vertrag definieren (Abstrakte Klasse)

```
package org.example.filesystem;

import java.util.Collection;

abstract class FileSystemElement {

    protected String name;

    FileSystemElement(String name) {
        this.name = name;
    }

    abstract long fetchSize();

    abstract boolean contains(FileSystemElement element);

    @Override
    public String toString() {
        return name;
    }

    abstract Collection<FileSystemElement> elements();
}
```

35

Den Vertrag definieren (Abstrakte Klasse)

Erinnern wir uns: Eine **abstrakte Klasse** ist wie eine Schablone oder ein Bauplan, der nicht vollständig ausgefüllt ist.

Keine Instanzierung: Wir können kein Objekt von `FileSystemElement` direkt erzeugen (`new FileSystemElement()` geht nicht). Es ist ein generisches Konzept. Es gibt in der Realität nur konkrete Files oder Directories.

Gemeinsamkeiten: Sie bündelt Eigenschaften, die alle teilen – hier zum Beispiel das Feld `name`.

Der Vertrag: Das Wichtigste für uns heute ist aber: Sie kann Methoden definieren, die die Kinder implementieren *müssen*.

Wir haben gesehen, wie wir die Größe berechnen (`fetchSize`) und wie wir Dinge suchen (`contains`). Die Übungsaufgabe verlangt nun eine Methode `elements()`, die uns **alle** Elemente liefert – das Element selbst und alles, was darin liegt. Da wir das Composite Pattern nutzen, müssen wir diesen 'Vertrag' in unserer obersten Klasse definieren.

Neuer Code: `abstract Collection<FileSystemElement> elements();`

Jedes Teil unseres Dateisystems muss in der Lage sein, eine Liste von sich und seinen Unter-Elementen zurückzugeben. Wir machen das abstrakt, weil File und Directory das völlig unterschiedlich handhaben werden. ´

Der einfache Fall – Das Blatt (File):

```
package org.example.filesystem;

import java.util.Collection;
import java.util.List;

class File extends FileSystemElement {

    private final long size;

    File(String name, long size) {
        super(name);
        this.size = size;
    }

    @Override
    long fetchSize() {
        return size;
    }

    @Override
    boolean contains(FileSystemElement element) {
        return false;
    }

    @Override
    Collection<FileSystemElement> elements() {
        // Gedankenprozess:
        // 1. Ich habe keine Kinder.
        // 2. Ich muss aber MICH SELBST zurückgeben.
        // 3. Die Rückgabe soll eine Collection sein.

        return List.of(this);
    }
}
```

36

Der einfache Fall – Das Blatt (File):

Fangen wir immer mit dem einfachsten Fall an: dem Abbruch der Rekursion bzw. dem Blatt im Baum. Das ist unsere File-Klasse. Ein File hat keine Kinder. Aber die Aufgabenstellung sagt: „Die Collection soll das Element selbst enthalten.“

Indem wir das Schlüsselwort **abstract** vor die Methode schreiben, zwingen wir alle Unterklassen (File und Directory), diese Methode zu implementieren. Wir nehmen ihnen die Entscheidung ab. Das führt uns direkt zum Konzept des **Overriding** (Überschreiben):

Was passiert? Die Unterklasse liefert ihre eigene, spezifische Version dieser Methode. Das File verhält sich anders als das Directory.

Die Annotation @Override: Wenn wir gleich in die Unterklassen gehen, schreiben wir **@Override** darüber. Das ist nicht nur Deko. Es ist eine Nachricht an den Compiler: 'Ich behaupte, ich überschreibe hier eine Methode der Elternklasse. Bitte prüf das für mich!'. Wenn wir uns beim Methodennamen vertippen würden, würde der Compiler dank **@Override** sofort schreien. Es ist also unsere Sicherheitsleine.

// Gedankenprozess:

// 1. Ich habe keine Kinder.

// 2. Ich muss aber MICH SELBST zurückgeben.

// 3. Die Rückgabe soll eine Collection sein.

Hier brauchen wir keine Rekursion. Ein File ist eine Sackgasse. Wir wickeln einfach this (das File selbst) in eine Liste ein. Fertig. Das ist unser Basisfall.

Der rekursive Fall – Der Container (Directory)

```
package org.example.filesystem;

import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedList;
import java.util.List;

class Directory extends FileSystemElement {

    // Rest der Klasse

    @Override
    Collection<FileSystemElement> elements() {
        // Wir erstellen eine leere, modifizierbare Liste.
        List<FileSystemElement> resultList = new ArrayList<>();

    }
}
```

37

Wir gehen in die Directory-Klasse. Ein Ordner ist ein Composite. Er enthält eine Liste von Kindern (elements), und diese Kinder können wieder Ordner sein. Hier brauchen wir unsere Rekursions-Strategie.

Die Liste vorbereiten Zuerst brauchen wir einen Ort, wo wir alle Ergebnisse sammeln. *Wir erstellen eine leere, modifizierbare Liste.*

Der rekursive Fall – Der Container (Directory)

```
@Override
Collection<FileSystemElement> elements() {
    // Wir erstellen eine leere, modifizierbare Liste.
    List<FileSystemElement> resultList = new ArrayList<>();
    // Schritt 1: Füge den Ordner selbst hinzu.
    resultList.add(this);
}
```

```
@Test
void dir1Elements() {
    var actual = dir1.elements();
    var expected = List.of(dir1, file1, dir2, dir3, file2, file3, dir4);
    assertIterableEquals(expected, actual);
}
```

38

Sich selbst hinzufügen Schauen wir auf den Unit-Test in der Aufgabe. Dort steht: `expected = List.of(dir1, file1, ...)` – das heißt, der Ordner (`dir1`) kommt als Erstes. Das nennt man Pre-Order Traversal.

Der rekursive Fall – Der Container (Directory)

```
@Override
Collection<FileSystemElement> elements() {
    // Wir erstellen eine leere, modifizierbare Liste.
    List<FileSystemElement> resultList = new ArrayList<>();
    // Schritt 1: Füge den Ordner selbst hinzu.
    resultList.add(this);
    // Schritt 2: Rekursiv durch alle Kinder gehen.
    for (FileSystemElement child : elements()) {
        // Der rekursive Aufruf!
        // Das Kind gibt uns eine Liste SEINER Inhalte.
        Collection<FileSystemElement> childContents = child.elements();

        // Wir fügen ALLES, was das Kind liefert, zu unserer Ergebnisliste hinzu.
        resultList.addAll(childContents);
    }
}
```

39

Die Rekursion (Die Magie): Jetzt müssen wir durch alle Kinder iterieren. Und hier ist der Trick: Wir müssen nicht wissen, ob das Kind eine Datei oder ein Ordner ist. Wir vertrauen einfach darauf, dass das Kind seine `elements()`-Methode korrekt implementiert hat (Polymorphie).

Das ist der entscheidende Punkt: **child.elements()**

Wenn das Kind ein **File** ist, kriegen wir eine Liste mit 1 Element zurück (Basisfall). Wenn das Kind ein **Directory** ist, startet dort der gleiche Prozess von vorne (Rekursion). Wir nutzen `addAll`, um die Teilergebnisse in unsere große Liste zu kippen."

Abschluss (Unmodifiable)

```
@Override
Collection<FileSystemElement> elements() {
    // Wir erstellen eine leere, modifizierbare Liste.
    List<FileSystemElement> resultList = new ArrayList<>();
    // Schritt 1: Füge den Ordner selbst hinzu.
    resultList.add(this);
    // Schritt 2: Rekursiv durch alle Kinder gehen.
    for (FileSystemElement child : elements) {
        // Der rekursive Aufruf!
        // Das Kind gibt uns eine Liste SEINER Inhalte.
        Collection<FileSystemElement> childContents = child.elements();

        // Wir fügen ALLES, was das Kind liefert, zu unserer Ergebnisliste hinzu.
        resultList.addAll(childContents);
    }
    // Schritt 3: Sicher zurückgeben.
    return Collections.unmodifiableCollection(resultList);
}
```

40

Schauen wir uns an, was wir gebaut haben:

File: Sagt einfach 'Hier bin ich'.

Directory: Sagt 'Hier bin ich' UND fragt dann jedes seiner Kinder 'Wer bist du und wen hast du dabei?'.

Das Ergebnis ist eine flache Liste, die den kompletten Baum ab diesem Punkt enthält. Genau das, was wir für Funktionen wie 'Alle Dateien in Unterordnern suchen' bräuchten."